Your WhatsApp AI Support Agent

A plain-English guide to what we built, how the code works,
and every bump we hit along the way (and how to fix them yourself next time)

What's in this guide:

1. The Big Picture

You now have a robot that lives on WhatsApp for a pretend online store called **AuraLoop Electronics**. Customers text it things like "*where is my order?*" or "*I want a refund*", and it answers using **real data** from a real database — never by guessing or making things up.

> **Think of it like a very well-trained new employee** at a call center, who:
>
> - Never says "I think your order shipped" — they always actually *look it up* first.
> - Knows the return policy by heart, but never bends the rules on their own — if a refund is expensive or unclear, they always call a manager over.
> - Writes down everything they did in a logbook, so if you ever ask "why did you say that?", there's a paper trail.

The "employee" is powered by **Google's Gemini AI**. But — and this is the most important idea in the whole project — **the AI is never allowed to decide the risky stuff on its own.** It can only:

1. Decide which "tool" (helper function) to use to look something up, and
2. Write the reply message.

Whether to actually *send* that reply, or instead hand the conversation to a real human, is decided by plain, boring, 100%-predictable computer code — not the AI. That's on purpose: AI can be confidently wrong sometimes, but simple "if this number is bigger than that number, call a human" logic never surprises you.

2. A Message's Journey (Step by Step)

Step	What Happens	In Plain Words
1. WhatsApp	Customer sends a message	Someone texts "where is my order?"

2. Twilio	Twilio (the WhatsApp middleman company) receives it and forwards it to our server	Like a mail carrier delivering the letter to our door
3. Identify the Customer	We look up who this phone number belongs to	Checking their name badge before helping them
4. Check the "explicit human request" rule	Plain code checks: did they just say "talk to a human"?	A simple keyword check — no AI needed for this
5. Gather context (RAG)	We fetch the last few messages, plus any relevant paragraphs from our policy document	Like handing the employee a sticky note with "here's what they said before, and here's the relevant page of the policy manual"
6. The AI Agent thinks	Gemini decides which tool(s) to call: look up the order? check refund eligibility?	The employee decides "let me check the computer system" instead of guessing
7. Tools run	Real database queries happen, real answers come back	The "computer system" actually gets checked
8. The AI drafts a reply	Gemini writes an answer using only the real data it just fetched	The employee writes their answer down
9. The Decision Engine judges	Plain code checks: is this risky? high value? low confidence?	A supervisor glancing over the employee's shoulder before it's sent
10a. Auto-reply	If it's safe, the reply is sent for real	Employee sends the message
10b. OR Escalate	If it's risky, a ticket is created and a human takes over — the AI's draft is thrown away, never sent	"Let me get my manager" — and the manager starts fresh
11. Everything is logged	Every tool call and every decision is written to the database	The logbook, so nothing is a mystery later

3. The Code Folders, Explained Simply

Here's every folder inside `app/`, explained like you're new to programming.

`app/db/models.py`

This is the **blueprint for the filing cabinet**. It describes every "drawer" (table) we have: customers, orders, products, tickets, conversations, messages, and two "logbook" drawers (tool_call_logs, audit_logs). It doesn't *do* anything by itself — it just describes the shape of the data.

`app/repositories/`

These are the **librarians**. Each one knows how to fetch or store things in one specific drawer. For example, `customer_repo.py` knows how to "find the customer whose phone number is X" or "create a new customer." Nothing else in the app is allowed to touch the database directly — everything goes through a librarian. This keeps things tidy and makes bugs easier to find.

`app/tools/`

These are the **specific tasks the AI is allowed to ask for help with** — and nothing more. There are five: look up my profile, look up an order, search products, create/list a ticket, and check refund eligibility. The AI can never do anything outside this list — it can't, say, delete a customer or look up someone else's order, because those abilities simply don't exist as tools.

`app/tools/refund_policy.py` is special: it's the actual **rulebook** for refunds (is the order still within 30 days? has it been delivered?), written as plain boring code with zero AI involved. The AI calls this rulebook and reports what it says — it never makes up its own answer.

`app/agent/`

This is the **AI's thinking loop**. `graph.py` is the core: it asks Gemini "what do you want to do?", and if Gemini says "call this tool," it runs the tool, shows Gemini the result, and asks again — looping until Gemini says "I'm ready to answer." Then a second, final question is asked: "okay, now give me your answer in this exact structured format" (so we can trust the reply has a confidence score, an intent label, etc., instead of just loose text).

`app/decision_engine/`

This is the **supervisor**. `rules.py` checks for phrases like "talk to a human." `engine.py` checks, in a fixed order: did they ask for a human? is the AI's confidence too low? is the order too expensive? is the refund situation unclear? Only plain "if/else" logic here — no AI, ever. This is the single most important file for *trust*: it's the one place that decides if a human needs to step in.

`app/services/`

These are the **connectors to the outside world**: `whatsapp.py` is the only file allowed to talk to Twilio (sending messages, checking that a message really came from Twilio and wasn't faked). `embeddings.py` talks to Gemini to turn text into a list of numbers (for searching the policy document by meaning, not just

keywords). `rag.py` does that policy search. `pipeline.py` is the "conductor" that calls everything else in the right order — it's the file that actually implements the 11-step journey from Section 2.

`app/routers/`

The **front door and windows** of the app. `webhook.py` is the front door Twilio knocks on when a WhatsApp message arrives. The other files (`conversations.py` , `tickets.py` , `escalations.py`) are "windows" a manager could look through later to see what happened — e.g. "show me every escalated conversation."

`app/knowledge_base/`

The actual **policy documents** (return policy, shipping policy, FAQs, and now your SmileCare Dental PDF too), plus `ingest.py` , which chops each document into bite-sized paragraphs and turns each one into a list of numbers Gemini can search by meaning later.

`scripts/`

`seed_data.py` invents 18 fake customers, 26 fake products, and 36 fake orders so we have something to test with. `simulate_message.py` lets us pretend a WhatsApp message arrived, without needing a real phone or Twilio webhook — useful for quick testing.

4. The Three Test Files, Explained

A "test" is just a small program that checks another piece of code does what it's supposed to, using made-up example situations. Think of it as a pop quiz we give our own code, so we find mistakes before a real customer does.

`tests/test_refund_policy.py`

This quizzes the refund *rulebook* (not the AI — the plain code rules). It makes up example orders like "delivered exactly 30 days ago" or "delivered 31 days ago" and checks the rulebook gives the right answer (eligible vs. not eligible) for each one. This is the kind of test that catches "off by one day" mistakes, which are very easy to accidentally get wrong.

`tests/test_decision_engine.py`

This quizzes the *supervisor*. It makes up pretend situations — "customer asked for a human," "AI wasn't confident," "order is very expensive" — and checks the supervisor escalates (or doesn't) correctly in each case, and in the right priority order. This is the test that protects you from the AI ever silently being allowed to approve something it shouldn't.

`tests/test_agent_graph.py`

This quizzes the AI's *thinking loop* itself — but using a "pretend AI" (a fake, scripted stand-in) instead of really calling Gemini. Why? Because real AI calls cost money and take time, and we just want to check our own wiring is correct: does the loop actually call the tool when asked? Does it correctly stop and finalize when the AI says it's done? Using a fake AI lets us check that in a fraction of a second, for free, every single time we change the code.

All three run in under 4 seconds total with the command `pytest -q`, and none of them need your real database or a real Gemini key — that's the point of tests like these: fast, free, repeatable checks of the parts we can fully control.

5. What is "ngrok" and a "Tunnel"? (The New Concept)

The problem: your laptop is sitting on your home/office network. Twilio's servers, out on the real internet, have no way to find or reach your laptop directly — it's like trying to mail a letter to someone with no street address. But Twilio needs to *send* your server a message every time someone texts your WhatsApp bot.

The solution: a "tunnel." A tunnel service gives your laptop a temporary, real, public internet address (like `https://jcxbksjh-8001.inc1.devtunnels.ms`), and quietly relays anything sent to that public address straight through to a specific port on your own machine (in our case, port 8001, where our app was listening). It's like a mail-forwarding service: the letter arrives at a public P.O. box, and is instantly re-delivered to your actual house.

ngrok is the most famous tool that does this. We tried to install it, but your antivirus (Windows Defender) flagged the ngrok program itself as suspicious and deleted it every time — three separate times, including

Microsoft's own official installer version. This is a known "false positive": tunneling tools get flagged sometimes because hackers also use them, even though ngrok itself is legitimate. We didn't try to force past your antivirus, since that's a security decision that's yours to make, not mine.

Instead, we used **VS Code's own built-in port forwarding** (the "Ports" panel), which does the exact same job as ngrok, but is Microsoft's own trusted tool built into the editor you're already using — so there was no antivirus conflict at all.

6. Every Error We Hit — and What It Really Meant

Software almost never works perfectly on the first try — that's completely normal, even for experienced developers. Here's everything that went wrong, in plain English, and how it was fixed.

Error 1: Database migration failed to connect (DNS error)
Supabase's "direct" database address only works over a newer kind of internet addressing (IPv6) that this network couldn't reach.

Fix: Used Supabase's alternate "session pooler" address instead, which works over the older, more universal addressing (IPv4). *Could you have caught this yourself?* Only with some networking knowledge — this one's genuinely tricky.

Error 2: Wrong database password
The very first password given didn't match what Supabase had on file.

Fix: You reset the password in Supabase's dashboard and gave me the new one. *Could you have caught this yourself?* Yes — the error message literally said "password authentication failed," which is a clear signal to double check the password.

Error 3: "type order_status already exists"
A technical mistake in how I told the database to set up its categories (like "delivered," "processing," etc.) — I accidentally told it to create the same category twice.

Fix: Changed the setup code to only create each category once. *Could you have caught this yourself?* Not really — this needed knowledge of a specific database library's quirks.

Error 4: "prepared statement already exists" (multiple times)
Supabase's shared, pooled database connections can be secretly recycled between different requests. Our database library was leaving "sticky notes" (prepared statements) with the same name on a connection, and the next request would find an old sticky note already there and complain.

Fix: Told our code to always use a random, never-repeating name for every "sticky note" instead of counting 1, 2, 3... *Could you have caught this yourself?* No — this is a genuinely obscure interaction between three different pieces of software.

Error 5: Orders saved with the wrong category text ("DELIVERED" instead of "delivered")
Our database library defaulted to sending the ALL-CAPS internal name of a category instead of the actual lowercase word we told the database to expect.

Fix: Explicitly told the library "always use the lowercase version." *Could you have caught this yourself?* Only by carefully reading the error message, which did clearly say what value it tried to save — so partially yes, with practice.

Error 6: Gemini said "429 you've used up your quota" (only 20 messages/day allowed)
The AI model you'd chosen has a very small free daily allowance.

Fix: Switched to a different Gemini model with a separate, more generous free allowance. *Could you have caught this yourself?* Yes, easily — the error message directly says "quota exceeded" and names the limit.

Error 7: Gemini said "model no longer available"
An older AI model version had been retired for new accounts.

Fix: The error message literally told us which newer model to use instead, so we switched. *Could you have caught this yourself?* Yes — the error message spells out the fix directly.

Error 8: "python-multipart library must be installed"
A required helper package for reading incoming web form data was missing.

Fix: Installed it with one command (`pip install python-multipart`). *Could you have caught this yourself?* Yes — this is one of the easiest kinds of error to fix: the message tells you exactly what's missing.

Error 9: "Twilio could not find a Channel with the specified From address"
We were trying to send WhatsApp messages "from" your regular Twilio phone number, but that number isn't set up for WhatsApp — only the special Sandbox number is.

Fix: Switched the "From" number to Twilio's fixed shared Sandbox number, `+14155238886` . *Could you have caught this yourself?* Somewhat — this needed knowing Twilio's Sandbox rules specifically.

Error 10: "Sandbox can only send to numbers that joined" (error 63015)
You hadn't yet texted the special "join <code>" message to the Sandbox number from your own WhatsApp.

Fix: You sent the join message, and it started working. *Could you have caught this yourself?* Yes, very easily — once told about the join step, which is standard for anyone setting up a Twilio Sandbox for the first time.

Error 11: Real WhatsApp messages got rejected with "403 Forbidden"
This was the trickiest one. Our server double-checks that every incoming message really came from Twilio (not an imposter) by re-computing a security signature and comparing it. But behind a tunnel, the server only sees its own private address (`localhost:8001`), not the public address Twilio actually used (`https://jcxbksjh-8001.inc1.devtunnels.ms`) — so the signatures never matched, and every real message was rejected.

Fix: Taught the server to read two special delivery headers the tunnel adds (`X-Forwarded-Proto` and `X-Forwarded-Host`) and reconstruct the real public address before checking the signature. *Could you*

> *have caught this yourself?* Not easily — this needed adding temporary debug logging to see exactly what the tunnel was sending, which is a normal but fairly advanced debugging technique.

7. Cheat Sheet: What You Can Fix Yourself Next Time

If you see this in the error message...	...it probably means
`password authentication failed`	Double-check the password in your `.env` file matches what's in Supabase/Twilio/etc.
`ModuleNotFoundError` or "library must be installed"	Run `pip install <the missing package name>`
`429` / "quota exceeded" / "RESOURCE_EXHAUSTED"	You've used up a free daily/monthly limit — wait, or switch to a different model/plan
`404` / "model not found" / "no longer available"	The AI model name is outdated — the error usually names the replacement
"could not find a Channel" / "not a valid phone number"	Check phone number formatting (needs `whatsapp:+` and country code) and that you're using the right Twilio number for WhatsApp
Sandbox error 63015 / "hasn't joined"	Re-send the "join <code>" message from your WhatsApp — it expires every 3 days
`403 Forbidden` on a webhook	A security signature check failed — usually means the server's idea of its own public address is wrong
Anything mentioning "prepared statement" or "enum type already exists"	These are deep database-library quirks — not worth debugging alone, just ask for help with the exact error text

Generated as a plain-English companion to the AI Customer Support & Sales Agent project. For the technical version of everything above, see `README.md` in the project folder.